

Lista 2 — Resolução Completa

Exercício 1 — Consultas com forall

```
?- forall(member(X,[1,2,3]),write(X)).
```

forall executa write(X) para cada X de [1,2,3], sem separador. **Resultado:** 123 seguido de true.

```
?- forall(between(1,5,I),(write(I*I),write(' '))).
```

write(I*I) **não calcula** o produto (não há is); apenas escreve o termo I*I na notação de operador infixo, com I já instanciado. **Resultado:** 1*1 2*2 3*3 4*4 5*5 seguido de true.

```
?- forall(between(10,20,I),write(I:' ')).
```

Mesma lógica: escreve o termo I:' ' (operador :) para cada I de 10 a 20. **Resultado:** 10: 11: 12: 13: 14: 15: 16: 17: 18: 19: 20: seguido de true.

Exercício 2 — Trace de a

```
a:-a(0).
```

```
a(X):- X>10,!.
```

```
a(X):- write(X),write(' '), X1 is X+1,a(X1).
```

```
?- a.
```

a(0) não satisfaz X>10, então cai na segunda cláusula: escreve 0, incrementa, chama a(1). Isso repete até X=11, quando 11>10 é verdadeiro e o corte ! interrompe, sem escrever 11.

Resultado: 0 1 2 3 4 5 6 7 8 9 10 seguido de true.

Exercício 3 — Pirâmide com wN/1

```
wN(0):-write(0),!.
```

```
wN(N):-write(N),N1 is N-1, wN(N1),write(N).
```

```
xxx(N) :- piramide(0,N).
```

```
piramide(I,N) :- I>N, !.
```

```
piramide(I,N) :-
```

```
    I=<N,
```

```
    K is N-I,
```

```

    espacos(K),
    wN(I),
    nl,
    I1 is I+1,
    piramide(I1,N).

```

```

espacos(0) :- !.
espacos(K) :- K>0, write(' '), K1 is K-1, espacos(K1).

```

Ideia: wN(N) escreve N, N-1, ..., 0, ..., N-1, N (efeito “espelho” por causa da recursão: escreve antes e depois da chamada recursiva). piramide/2 cuida da indentação, colocando N-I espaços antes de cada linha I.

```

?- xxx(3).
    0
   101
  21012
 3210123
true.

```

Exercício 4 — Potenciação com acumulador (só + - *)

```

potencia(_, 0, 1) :- !.
potencia(X, Y, P) :- Y > 0, potencia_acc(X, Y, 1, P).

```

```

potencia_acc(_, 0, Acc, Acc) :- !.
potencia_acc(X, Y, Acc, P) :-
    Y > 0,
    Acc1 is Acc * X,
    Y1 is Y - 1,
    potencia_acc(X, Y1, Acc1, P).

```

```

?- potencia(2, 5, X).
X = 32.

```

Ideia: Acc carrega o produto parcial (começa em 1). A cada chamada multiplica por X e decrementa Y, até Y=0, quando Acc já é o resultado final.

Exercício 5 — Trace de findall

```

d(0).
d(1).

?- findall([A,B],(d(A),d(B)),L).

```

O objetivo $(d(A), d(B))$ é satisfeito para **todas** as combinações de A e B (produto cartesiano), na ordem em que o Prolog encontra por backtracking:

Resultado: L = $[[0,0], [0,1], [1,0], [1,1]]$

Exercício 6 — palindromo/1

```
palindromo(L) :- reverso(L, L).
```

```
reverso([], []).
```

```
reverso([H|T], R) :- reverso(T, RT), append(RT, [H], R).
```

Ideia: uma lista é palíndromo se ela é igual à sua própria inversa.

```
?- palindromo([a,b,c,d,c,b,a]).  
true.
```

```
?- palindromo([a,b,c]).  
false.
```

Exercício 7 — metIguais/1 (com append)

```
metIguais(L) :-  
    length(L, Len),  
    Len mod 2 == 0,  
    Half is Len // 2,  
    length(A, Half),  
    append(A, B, L),  
    A == B.
```

Ideia: calcula o tamanho total, exige que seja par, fixa o tamanho da primeira metade (`length(A,Half)` restringe o `append` a cortar exatamente na metade) e compara as duas partes.

```
?- metIguais([a,b,c,a,b,c]).  
true.
```

```
?- metIguais([a,b,c,a,b,d]).  
false.
```

Exercício 8 — insOrd/3

```
insOrd(X, [], [X]).
```

```
insOrd(X, [H|T], [X,H|T]) :- X <= H, !.
```

```
insOrd(X, [H|T], [H|R]) :- X > H, insOrd(X, T, R).
```

Regra base: lista vazia $\rightarrow$ insere X sozinho. Regra recursiva: se X já é menor ou igual à cabeça, insere na frente; senão, mantém a cabeça e insere recursivamente na cauda.

```
?- insOrd(4, [2,3,5,7], L).  
L = [2,3,4,5,7].
```

Exercício 9 — particiona/3

```
particiona(L, L1, L2) :- particiona(L, L1, L2, esq).
```

```
particiona([], [], [], _).  
particiona([H|T], [H|L1], L2, esq) :- particiona(T, L1, L2, dir).  
particiona([H|T], L1, [H|L2], dir) :- particiona(T, L1, L2, esq).
```

Ideia: alterna cada elemento entre as duas listas de saída (esquerda/direita). Se o número total de elementos for ímpar, a primeira lista (L1) fica com um elemento a mais, pois começamos alternando por ela.

```
?- particiona([1,2,3,4,5], L1, L2).  
L1 = [1,3,5], L2 = [2,4].
```

Exercício 10 — merge/3

```
merge([], L, L) :- !.  
merge(L, [], L) :- !.  
merge([H1|T1], [H2|T2], [H1|R]) :- H1 <= H2, !, merge(T1, [H2|T2], R).  
merge([H1|T1], [H2|T2], [H2|R]) :- H1 > H2, merge([H1|T1], T2, R).  
  
?- merge([a,b,b,k,z], [c,m,n,o], X).  
X = [a,b,b,c,k,m,n,o,z].
```

(Compara alfabeticamente com @=</@> seria mais correto para átomos, mas =</> funciona igual aqui pela ordem padrão do Prolog para átomos.)

Exercício 11 — Bug no max/3

```
max(X,Y,M) :- !, X > Y, M = X.  
max(X,Y,M) :- !, X <= Y, M = Y.
```

O erro: o corte ! está **antes** do teste X > Y, e não depois. Isso significa que o Prolog se compromete com a primeira cláusula **antes mesmo de saber se ela**

vai funcionar. Se $X > Y$ falhar depois do corte, não há mais como voltar atrás para tentar a segunda cláusula — a consulta inteira falha.

- $?- \text{max}(4,3,M).$ → primeira cláusula, corte, $4 > 3$ é verdadeiro → $M=4$.
Funciona por sorte (a condição da 1ª cláusula era mesmo verdadeira).
- $?- \text{max}(3,4,M).$ → primeira cláusula, corte, $3 > 4$ é **falso** → como o corte já eliminou a alternativa da segunda cláusula, a consulta **falha** (**false.**), em vez de retornar $M=4$.

Correção:

```
max(X,Y,X) :- X > Y, !.
```

```
max(X,Y,Y) :- X <= Y.
```

Aqui o corte só é executado **depois** que a condição já foi confirmada, então ele apenas evita tentar a segunda cláusula desnecessariamente — sem cortar uma alternativa válida.

Exercício 12 — Corte incondicional em p

```
a. b.  
p:-!,a.  
p:-b.
```

```
?- b.
```

A pergunta $?- b.$ não usa p , então simplesmente tem sucesso (b é fato). Mas o ponto do exercício é entender o que acontece se perguntarmos $?- p.$ (com possível backtracking):

p tenta a primeira cláusula: o corte $!$ é executado **imediatamente** (incondicionalmente, pois não há nada antes dele para falhar), o que **elimina para sempre a alternativa $p:-b.$** Em seguida chama a , que tem sucesso. Então $?- p.$ retorna **true**.

Se pedirmos mais soluções (;), o Prolog **não tenta $p:-b$** — mesmo b sendo verdadeiro, ele nunca é avaliado nessa prova, porque o corte já “trancou” a escolha na primeira cláusula.

Exercício 13 — pred1, pred2, pred3 com findall

```
pred1(L, R) :- findall([X], member(X, L), R).  
pred2(L, R) :- findall(pred(X,X), member(X, L), R).  
pred3(L, R) :- findall([element,X], member(X, L), R).  
  
?- pred1([a,b,c,d,e],L).  
L = [[a],[b],[c],[d],[e]].
```

```
?- pred2([a,b,c,d,e],L).
L = [pred(a,a),pred(b,b),pred(c,c),pred(d,d),pred(e,e)].

?- pred3([a,b,c,d,e],L).
L = [[element,a],[element,b],[element,c],[element,d],[element,e]].
```

Exercício 14 — Problema dos jarros (3 e 4 litros)

Estado $[X,Y]$: X = conteúdo do jarro de 3L, Y = conteúdo do jarro de 4L.

```
transforma('encher o jarro 1', [X,Y], [3,Y]) :- X < 3.
transforma('encher o jarro 2', [X,Y], [X,4]) :- Y < 4.

transforma('esvaziar o jarro 1', [X,Y], [0,Y]) :- X > 0.
transforma('esvaziar o jarro 2', [X,Y], [X,0]) :- Y > 0.

% transferir do jarro 1 (cap. 3) para o jarro 2 (cap. 4)
% caso 1: tudo cabe no jarro 2
transforma('transferir do jarro 1 para o 2', [X,Y], [0,Z]) :-
    X > 0, Y < 4, Z is X + Y, Z <= 4.
% caso 2: jarro 2 enche e sobra água no jarro 1
transforma('transferir do jarro 1 para o 2', [X,Y], [R,4]) :-
    X > 0, Y < 4, Z is X + Y, Z > 4, R is Z - 4.

% transferir do jarro 2 (cap. 4) para o jarro 1 (cap. 3)
% caso 1: tudo cabe no jarro 1
transforma('transferir do jarro 2 para o 1', [X,Y], [Z,0]) :-
    X < 3, Y > 0, Z is X + Y, Z <= 3.
% caso 2: jarro 1 enche e sobra água no jarro 2
transforma('transferir do jarro 2 para o 1', [X,Y], [3,R]) :-
    X < 3, Y > 0, Z is X + Y, Z > 3, R is Z - 3.
```

Isso segue exatamente o padrão `oper(ação, estado, estado')` visto no slide do “problema dos jarros” do material (jarro de 3L e 4L), adaptado para o nome `transforma`.

Uma solução manual (passo a passo) partindo de $[0,0]$ até obter $[_ ,2]$:

```
[0,0]
--(encher jarro 2)--> [0,4]
--(transferir 2 para 1)--> [3,1]
--(esvaziar jarro 1)--> [0,1]
--(transferir 2 para 1)--> [1,0]
--(encher jarro 2)--> [1,4]
--(transferir 2 para 1)--> [3,2]    <- META: jarro de 4L tem 2 litros
```

Um predicado de busca (não pedido no enunciado, mas útil para verificar) usando busca em profundidade com lista de visitados:

```
resolve(Estado, Estado, _, []) :- meta(Estado).
resolve(Estado, Meta, Visitados, [Acao|Resto]) :-
    transforma(Acao, Estado, Proximo),
    \+ member(Proximo, Visitados),
    resolve(Proximo, Meta, [Proximo|Visitados], Resto).

meta([_,2]).

% ?- resolve([0,0], _, [[0,0]], Caminho).
```